

데이터 아키텍처 설계 문서

사내 생성형 AI 게이트웨이 — secure-prompt-gateway

담당 역할	Data Architect (DA)
작성자	조종원
문서 버전	v1.0
작성일	2026-09-03
대상 시스템	PostgreSQL 16 · Flyway · Spring Data JPA
단일 진실 원천	사내_AI_게이트웨이_기획서_v1.0

목차

산출물 안내

- 1. 데이터 요구사항 정의서
- 2. ERD 설계서
- 3. 스키마 설계서
- 4. 마이그레이션 설계서
- 5. 엔티티·리포지토리 매핑 설계서
- 6. 요구사항 추적 매트릭스
- 7. 스키마 검증 보고서

01~07은 설계가 실제로 진행되어야 하는 순서다. 각 장 끝의 "다음 단계로 넘기는 것"이 다음 장의 입력이 되며, 07장이 그 사슬 전체를 실제 데이터베이스에서 검증한다.

데이터 아키텍처 산출물 (Data Architect)

이 폴더는 DB 설계가 어떤 순서로, 무엇을 근거로 만들어졌는지를 복원한 문서 묶음이다.

단일 진실 원천(SSOT)은 여전히 `사내_AI_게이트웨이_기획서_v1.md` 이고, 실행되는 정답은 `backend/src/main/resources/db/migration/V*.sql` 이다. 이 폴더는 그 둘 사이의 **판단 과정**을 남긴다.

왜 이 폴더가 필요한가

구현이 먼저 완성된 뒤에 문서를 쓰면, 설계가 결과에서 역산된 것처럼 보인다. 그러면 발표에서 "왜 이 테이블입니까?"라는 질문에 "그렇게 만들어져 있습니다"밖에 답하지 못한다.

DA의 작업은 원래 다음 순서다. 이 폴더는 각 단계를 **그 단계에서 알 수 있었던 것만으로** 재구성했다.

①	요구사항 정의	"무엇을 저장해야 하는가"	← 기획서 3·4·5·7장에서 도출
↓			
②	ERD 설계	"어떤 개체와 관계로 나눌 것인가"	← 정규화·카디널리티 결정
↓			
③	스키마 설계	"어떤 타입·제약으로 못 박을 것인가"	← 물리 모델
↓			
④	마이그레이션	"어떻게 재현 가능하게 적용할 것인가"	← Flyway V1·V2·V3
↓			
⑤	엔티티/리포지토리	"애플리케이션이 어떻게 읽고 쓰는가"	← JPA 매핑
↓			
⑥	추적/검증	"요구사항이 실제로 반영되었는가"	← RTM + 검증 실행

문서 목록

#	문서	답하는 질문	발표 활용
01	데이터 요구사항 정의서	무엇을 저장해야 하는가 — 데이터 요구사항 22건	3분 섹션 1 — 서비스 기획
02	ERD 설계서	왜 8개 테이블로 나눴는가. 관계와 정규화 근거	4분 섹션 3 — 시스템 아키텍처 & 설계
03	스키마 설계서	타입·제약·인덱스를 왜 그렇게 정했는가	4분 섹션 3 / Q&A
04	마이그레이션 설계서	V1·V2·V3의 경계와 재작업 규칙	4분 섹션 4 — Scaffolding
05	엔티티·리포지토리 매핑 설계서	DDL과 JPA가 어긋나지 않는다는 보장	4분 섹션 4 / Q&A
06	요구사항 추적 매트릭스	요구사항 하나가 어디까지 이어지는가	Q&A 대비 (누락 방어)
07	스키마 검증 보고서	실제로 맞는가 — 실행 결과	Q&A 최강 카드

표기법 — 이 묶음에서 쓰는 코드 세 가지

문서 안에 `DR-01` , `UC-03` , `D14` 같은 코드가 나온다. 출처가 서로 다르다. 본문에서는 항상 `DR-01` · 원문/전송본 분리 처럼 이름을 붙여 쓰므로 이 표로 돌아올 일은 거의 없다.

코드	뜻	누가 정했나	원본
UC-nn	Use-Case. 사용자가 무엇을 하는가	기획서에 원래 있던 것	기획서 3.2
Dn	Decision. 기획서의 모순·미정 항목을 어떻게 결정했는가	팀이 2026-09-02에 결정	기획서 0.5
DR-nn	Data Requirement. 무엇을 저장해야 하는가	DA가 이 묶음에서 새로 붙인 번호	이번 문서

`DR` 은 기획서에 없다. Use-Case는 "사용자가 무엇을 하는가"이지 "무엇을 저장해야 하는가"가 아니어서, 기획서 3·4·5·7장에서 데이터 관점으로 다시 도출해 번호를 붙였다. 번호가 있어야 **06번 추적 매트릭스**에서 **요구사항** → **컬럼** → **마이그레이션** → **엔티티** → **API/화면**을 한 줄로 이을 수 있다.

발표 주의: `UC` 와 `D` 는 팀 공용어지만 **`DR` 은 이 묶음 안에서만 쓰는 말이다.** 다른 조원이나 교수님은 `DR-07` 이 뭔지 모른다. 말할 때는 번호 대신 이름을 쓴다.

DR — 데이터 요구사항 22건

DR-01 원문/전송본 분리	DR-02 검사 1회=1레코드	DR-03 근거를 항목 단위로	DR-04 근거의 출처 구분
DR-05 AI 후보는 확정 전 무효	DR-06 사람의 확정 기록	DR-07 비동기 AI 상태 추적	DR-08 상태↔판정 무모순
DR-09 기준은 코드가 아니라 데이터	DR-10 부서별 적용 정책	DR-11 소유 부서≠적용 부서	DR-12 규칙의 유효기간
DR-13 규칙 코드가 공통 식별자	DR-14 마스킹 라벨은 MASK에만	DR-15 정책 버전 시점 보존	DR-16 감사 목록 필터·정렬
DR-17 span은 원문 기준	DR-18 로그인 없는 사용자 식별	DR-19 스키마 변경 없이 AI 응답 보관	DR-20 값 집합 DB 강제
DR-21 시드에 개인정보 없음	DR-22 확장 지점 논리 표기		

UC — Use-Case (기획서 3.2)

UC-01 프롬프트 제출·규칙 판정	UC-02 마스킹 확인·재제출	UC-03 AI 판정 요청·비동기
UC-04 판정 폴링	UC-05 감사 목록 조회	UC-06 AI 제안 확정
UC-07 부서 정책 조회	UC-08~10 출력검사·첨부·정책편집 (Future)	

D — 이 묶음에서 인용하는 결정 (기획서 0.5)

D1 중첩 억제	D2 INFOSEC 부서 코드	D3 span 재계산 안 함
D5 BLOCK이면 마스킹 안 함	D6 review_status 4값	D7 PENDING_REVIEW 본문 보존
D9 finding은 규칙당 1건	D11 Case A 원시4→억제2	D14 BLOCK⇒NULL은 필요조건
D17 파일 입력은 기존 경로	D19 owner_dept_id 신설	D20 embargo_until 신설

기획서 절 번호

기획서 6.2 · 테이블 상세처럼 이름을 붙여 쓴다. 자주 인용되는 것만 적는다.

0.3 범위 상한	4장 책임 경계	5.4 SCR-02 감사 콘솔
6.2 테이블 상세	6.3 관계 정의	6.4 설계 근거
7.1 정책 4종	7.2 규칙 10종	7.3 부서 매트릭스
7.4 판정 절차	7.5 충돌 해결	7.6 마스킹 규칙
8.4 엔드포인트 상세	10.5 감사 로그 시드	11.3 설정 분리

다른 산출물과의 관계

파일	관계
사내_AI_게이트웨이_기획서_v1.md 6·7장	입력. 이 폴더는 6·7장을 근거로 삼는다. 충돌 시 기획서가 정답
backend/.../V1__schema.sql · V2__seed.sql · V3__embargo.sql	출력. 실행되는 정답. 문서와 어긋나면 DDL이 정답
docs/erd.dbml	dbdiagram.io 입력용 소스. 02번 문서의 도해가 여기서 나온다
_workspace/02_data-architect_schema-notes.md	작업 중 실무 노트(막힌 것·해결). 이 폴더는 그것을 정리한 결과물
_workspace/01_data-architect_names.md	컬럼명·enum 값의 확정 목록 (계약)

발표에서 쓸 한 문장

"정책과 임계값은 코드가 아니라 DB에 있습니다. 규칙 하나를 바꾸면 배포 없이 판정이 바뀌고, 그 판정이 어떤 정책 **버전**으로 내려졌는지는 inspection.policy_snapshot 에 시점 보존됩니다."

이 문장이 성립하는 근거가 02번(설계)과 07번(검증)에 있다.

01. 데이터 요구사항 정의서

작성 근거: 기획서 2장(문제 정의) · 3장(Use-Case) · 4장(책임 경계) · 5장(화면) · 7장(정책·규칙) · 0.5절(결정 D1 · 중첩 억제~D20 · embargo_until 신설) 이 문서의 위치: ERD를 그리기 전에 확정되어야 하는 것. "무엇을 저장해야 하는가"만 다루고, 테이블 이름은 아직 정하지 않는다.

0. 도출 방법

데이터 요구사항은 상상해서 만들지 않는다. 세 갈래에서만 나온다.

갈래	질문	기획서 근거
Use-Case	이 행위가 끝났을 때 무엇이 남아야 하는가	3.2 UC-01~UC-07
화면	이 화면이 그리려면 무엇을 읽어야 하는가	5.3 SCR-01, 5.4 SCR-02
규칙·정책	판단의 기준이 어디에 있어야 하는가	7.1~7.6, 4장 책임 경계

여기에 가이드의 AI-Ready 4대 원칙이 비기능 요구사항으로 얹힌다.

1. 기능 데이터 요구사항

1.1 프롬프트와 판정 (UC-01 · 프롬프트 제출·규칙 판정 ~ UC-04 · 판정 폴링)

ID	요구사항	유래	이유
DR-01	사용자가 입력한 원문과 외부로 나가는 전송본을 분리해 보관한다	UC-01(프롬프트 제출·규칙 판정), UC-02(마asking 확인·재제출), 2.4	원문은 감사 증적이고 전송본은 마asking 결과다. 한 컬럼에 덮어쓰면 "무엇이 가려졌는가"를 사후에 증명할 수 없다
DR-02	검사 1회의 결과가 하나의 레코드로 남는다	UC-01(프롬프트 제출·규칙 판정)	화면·API·감사 목록이 모두 "판정 1건"을 단위로 다룬다. 프롬프트와 분리해야 출력 검사(UC-08 · 출력 검사, Future Future)를 같은 구조로 붙일 수 있다
DR-03	판정의 근거를 항목 단위로 남긴다 (어떤 규칙이 어디에 걸렸는가)	UC-01(프롬프트 제출·규칙 판정), UC-06(AI 제안 확정), 2.4	상태만 남기면 "왜 차단됐는지"를 담당자가 재현할 수 없다. 감사 시스템의 존재 이유가 여기다
DR-04	근거 항목은 출처(규칙 / AI)를 구분할 수 있어야 한다	4장 책임 경계	규칙은 확정 판정이고 AI는 제안이다. 같은 목록에 섞이되 성격이 다르므로 구분 축이 필요하다
DR-05	AI가 만든 근거 항목은 사람의 확정 전에는 효력이 없다	4장, D6(review_status 4값)	"AI는 결정하지 못한다"가 이 서비스의 핵심 주장이다. 애플리케이션 코드로만 지키면 코드를 우회한 쓰기 한 번에 무너진다
DR-06	사람이 확정한 사실(누가·언제)을 남긴다	UC-06(AI 제안 확정), 4장	사후 소명의 대상이 사람이다. decidedBy=HUMAN 과 담당자 식별이 함께 있어야 한다
DR-07	AI 판정은 비동기이므로 진행 상태를 추적할 수 있어야 한다	UC-03(AI 판정 요청·비동기), UC-04(판정 폴링), 8.5	202 + 폴링 흐름. 대기중 · 완료 · 실패 · 호출안함 이 구분되지 않으면 화면이 무엇을 그럴지 결정하지 못한다
DR-08	판정 결과와 프롬프트의 상태가 서로 모순되지 않아야 한다	7.5 충돌 해결	BLOCKED 인데 판정이 ALLOW 인 행이 생기면 감사 데이터 전체의 신뢰가 깨진다

1.2 정책과 규칙 (UC-07 · 부서 정책 조회, 7장)

ID	요구사항	유래	이유
DR-09	판단의 기준(패턴·엑션·심각도)은 코드가 아니라 데이터로 존재한다	11.3, 7.2	규칙 하나 바꾸는 데 배포가 필요하면 "정책 게이트웨이"가 아니라 그냥 하드코딩된 필터다
DR-10	정책은 부서마다 다르게 적용된다. 전사 공통 정책과 부서 한정 정책이 공존한다	7.3 매트릭스, UC-07(부서 정책 조회)	개발팀에는 P-CONF가 적용되지 않는다는 것이 데모 Case B/C의 근거다. 이 차이를 표현하지 못하면 시연이 성립하지 않는다
DR-11	정책을 만든 부서와 적용받는 부서는 다를 수 있다	D19(owner_dept_id 신설), 7.1	엠바고는 홍보팀이 걸고 개발팀·영업팀이 걸린다. 두 개념을 한 축으로 합치면 "홍보팀이 자기 엠바고에 걸린다"는 모순이 생긴다
DR-12	규칙에는 유효기간이 있을 수 있다	D20(embargo_until 신설), 7.2	엠바고는 해제일이 지나면 자동으로 풀려야 한다. 날짜를 코드나 화면에 박으면 DR-09(기준은 코드가 아니라 데이터)가 무너진다
DR-13	규칙의 식별 코드가 화면·로그·API·테스트에서 동일하게 쓰인다	7.2, 계약서 §4	PII-RRN-01 이 발표 대사이자 응답 필드이자 테스트 기대값이다. 전역 유일해야 한다
DR-14	마asking 라벨은 마asking 엑션 규칙에만 존재한다	D5(BLOCK이면 마asking 안 함), 7.5	차단 규칙에 라벨이 없는데 마asking가 호출되면 실행 시점에 죽는다. 데이터 단계에서 막아야 한다

1.3 감사와 시점 보존 (UC-05 · 감사 목록 조회, SCR-02)

ID	요구사항	유래	이유
DR-15	판정이 어떤 정책 버전으로 내려졌는지 시점 보존한다	6.4, 8.4	정책은 나중에 바뀐다. 지금 정책으로 과거 판정을 재해석하면 감사가 성립하지 않는다
DR-16	감사 목록을 부서·상태·기간으로 필터하고 최신순으로 정렬한다	5.4 SCR-02	조회 패턴이 고정이므로 인덱스 설계의 근거가 된다
DR-17	근거 항목의 위치(오프셋)는 원문 기준으로 저장하고 재계산하지 않는다	D3(span 재계산 안 함)	마스킹으로 길이가 변하므로 전송본 기준으로는 위치가 어긋난다. 화면 하이라이트는 라벨 문자열 검색으로 처리한다

1.4 사용자와 부서 (0.3 범위 제약)

ID	요구사항	유래	이유
DR-18	로그인 없이 사용자를 식별한다	0.3, 8.1	인증은 범위 밖. 요청 헤더의 사용자 식별자로 부서를 찾고, 부서가 적용 정책을 결정한다

2. 비기능 데이터 요구사항

ID	요구사항	유래	이유
DR-19	AI 응답 원본을 스키마 변경 없이 보관한다	AI-Ready · Structured Data	실제 LLM으로 교체하면 응답 필드가 늘어난다. 그때마다 컬럼을 추가하면 "AI-Ready"라는 주장이 거짓이 된다
DR-20	값 집합(enum 성격)은 DB 수준에서 강제한다	4장, D6(review_status 4 값)	애플리케이션에만 두면 시드 SQL·수동 패치·다른 컴포넌트의 직접 쓰기로 오염된다. 감사 데이터에서 그런 조용한 오염이다
DR-21	시드 데이터에 실제 개인정보 형태 문자열을 넣지 않는다	10.1, 10.5	개인정보 보호를 주제로 한 서비스가 자기 시드에 주민번호를 넣으면 발표에서 그 자리에서 지적당한다
DR-22	범위 밖 확장 지점을 논리 모델로 표기한다 (DDL은 만들지 않는다)	0.3, 6.5, 17장	확장성을 주장하려면 자리를 그려야 하고, 범위를 지키려면 실행하지 않아야 한다. 두 요구를 동시에 만족하는 방법이 Logical-only 표기다

3. 명시적으로 요구하지 않는 것

범위 상한(기획서 0.3(범위 상한))이 데이터 모델에도 그대로 적용된다. 아래는 만들지 않는다.

제외 항목	근거	확장 시 위치
계정·비밀번호·권한 테이블	0.3 — 로그인 미구현	17장
첨부파일 바이너리 저장	0.3 — 프론트에서 텍스트 추출 후 기존 경로로 전송 (D17 · 파일 입력은 기존 경로)	attachment (Future)
정책 변경 이력 테이블	0.3 — 정책 편집 UI 없음. policy.version + 스냅샷으로 대체	policy_audit (Future)
출력(AI 응답) 검사 결과	0.3 — 입력 검사만	inspection.phase='OUTPUT' 자리만 확보
모델 파라미터 테이블	11.3 — 현재는 환경변수 (Config Isolation)	ai_provider_config (Future)

inspection.phase 처럼 컬럼은 만들되 값은 하나만 쓰는 방식이 요구사항 DR-22(확장 지점 논리 표기)의 구현 형태다. 테이블을 미리 만들면 범위 위반이고, 자리조차 없으면 확장성 주장이 근거를 잃는다.

4. 다음 단계로 넘기는 것

이 문서는 테이블 이름을 정하지 않았다. 그것이 02번 ERD 설계서의 일이다.

넘기는 질문: 1. DR-01(원문/전송본 분리)의 원문·전송본은 한 테이블인가 두 테이블인가 2. DR-03(근거를 항목 단위로)의 근거 항목을 규칙 출처와 AI 출처로 나눌 것인가 합칠 것인가 (DR-04 · 근거의 출처 구분과 상충하는가) 3. DR-10(부서별 적용 정책)의 부서×정책을 어떤 카디널리티로 표현할 것인가 4. DR-15(정책 버전 시점 보존)의 시점 보존을 이력 테이블로 할 것인가 스냅샷으로 할 것인가 5. DR-19(스키마 변경 없이 AI 응답 보관)의 "스키마 변경 없이"를 어떤 타입으로 달성할 것인가

02. ERD 설계서

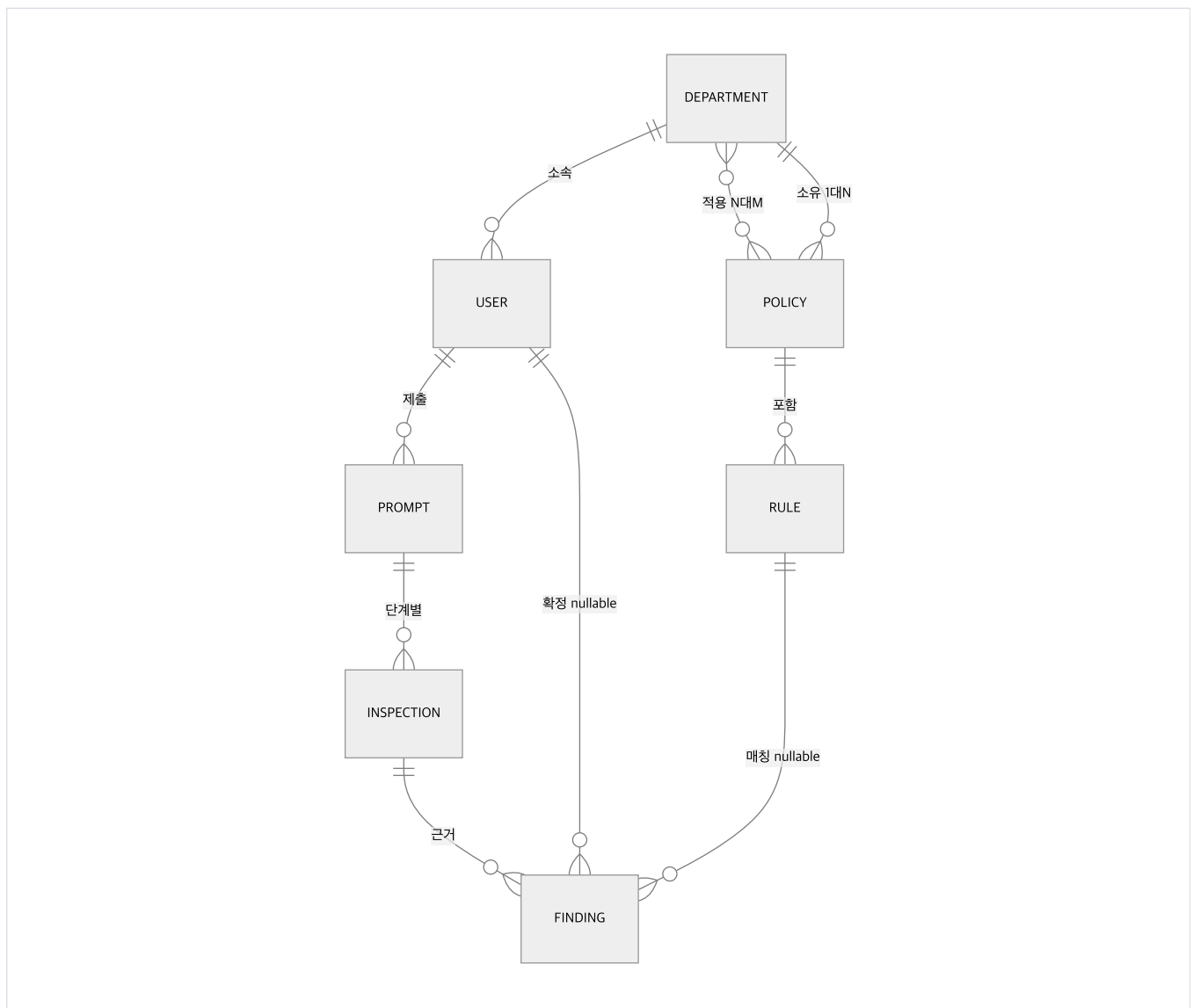
입력: 01 데이터 요구사항 정의서 DR-01~DR-22 출력: docs/erd.dbml (dbdiagram.io 소스) → 이 문서의 도해 이 문서의 위치: 요구사항을 개체와 관계로 옮기는 단계. 타입·제약은 아직 정하지 않는다 (03번의 일).

1. 개체 도출 — 요구사항에서 명사를 뽑는다

요구사항	문장에서 반복되는 명사	개체 후보
DR-01(원문/전송본 분리), DR-18(로그인 없는 사용자 식별)	사용자가 입력한 프롬프트	사용자, 프롬프트
DR-02(검사 1회=1레코드), DR-07(비동기 AI 상태 추적)	검사 1회의 결과	검사
DR-03(근거를 항목 단위로), DR-04(근거의 출처 구분), DR-05(AI 후보는 확정 전 무효)	판정의 근거 항목	발견 항목
DR-09(기준은 코드가 아니라 데이터), DR-13(규칙 코드가 공통 식별자), DR-14(마스킹 라벨은 MASK에만)	규칙의 패턴·액션	규칙
DR-10(부서별 적용 정책), DR-11(소유 부서≠적용 부서)	부서마다 다른 정책	정책, 부서

여섯 개다. 여기에 DR-10(부서별 적용 정책)의 부서×정책 다대다를 표현할 **연결 개체**가 하나 더 붙어 7개, DR-03(근거를 항목 단위로)을 검사와 분리하면서 8개가 된다.

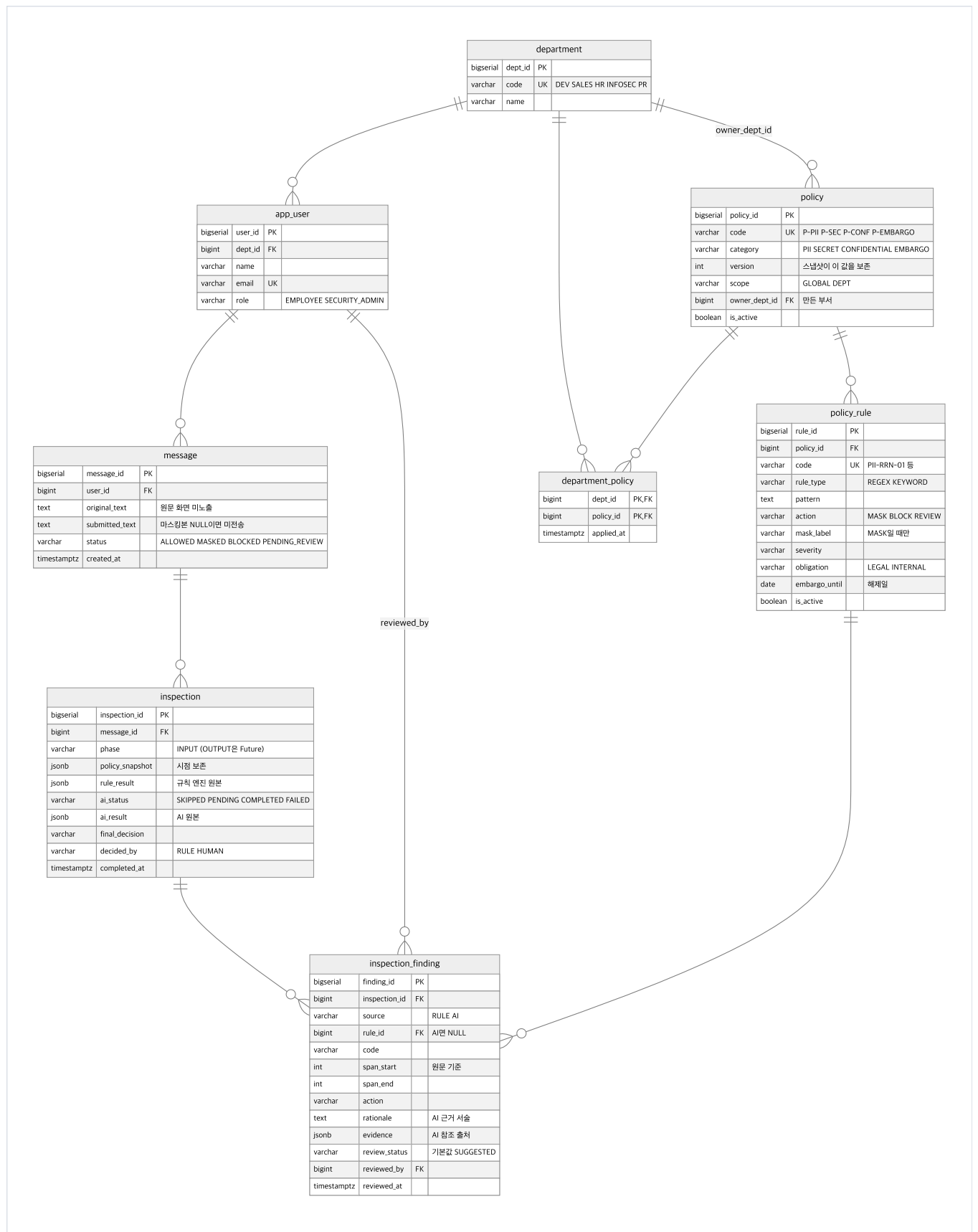
2. 개념 ERD



개념 개체	우리말	이후 테이블명
DEPARTMENT	부서	department
USER	사용자	app_user
POLICY	정책	policy
RULE	규칙	policy_rule
PROMPT	프롬프트	message
INSPECTION	검사	inspection
FINDING	발견 항목	inspection_finding

부서와 정책 사이에 선이 두 개인 것이 이 모델의 특징이다 — DR-11(소유 ≠ 적용)의 직접적 결과다.

3. 논리 ERD (Core 8)



dbdiagram.io용 소스는 docs/erd.dbml 이다. 발표용 PNG는 그 파일을 붙여넣어 export한다.

4. 관계 정의

#	관계	카디널리티	선택성	근거
R1	department — app_user	1:N	사용자는 부서 필수	부서가 적용 정책을 결정한다(DR-10 · 부서별 적용 정책). 부서 없는 사용자는 판정 불가
R2	department — policy (적용)	N:M → department_policy	양쪽 선택	DR-10(부서별 적용 정책)
R3	department — policy (소유)	1:N	소유 부서는 선택	DR-11(소유 부서≠적용 부서). 기존 3개 정책은 나중에 채워졌다
R4	policy — policy_rule	1:N	규칙은 정책 필수	정책 없는 규칙은 적용 범위를 결정할 수 없다
R5	app_user — message	1:N	필수	DR-18(로그인 없는 사용자 식별)
R6	message — inspection	1:N	필수	phase별 1건. 현재 INPUT만이라 사실상 1:1이나, OUTPUT 확장(DR-22 · 확장 지점 논리 표기)을 위해 1:N
R7	inspection — inspection_finding	1:N	0건 가능	위반이 없으면 findings는 빈 배열
R8	policy_rule — inspection_finding	1:N	nullable	DR-04(근거의 출처 구분). AI 후보는 규칙을 참조하지 않는다
R9	app_user — inspection_finding	1:N	nullable	DR-06(사람의 확장 기록). 미검토 항목은 확정자가 없다

R8-R9의 nullable이 이 설계의 핵심이다. 두 FK가 NULL일 수 있다는 사실이 "규칙 근거"와 "AI 후보"를 한 테이블에 담으면서도 성격을 구분하는 방법이다.

5. 설계 결정 — 01번이 넘긴 5개 질문에 답한다

Q1. 원문과 전송본은 한 테이블인가 두 테이블인가 (DR-01 · 원문/전송본 분리)

한 테이블, 두 컬럼. message.original_text / message.submitted_text

대안	기각 사유
별도 masked_message 테이블	1:1 관계를 테이블로 나누면 조화가 항상 조인이다. 두 값의 생명주기가 동일(같은 트랜잭션에서 생기고 함께 사라진다)하므로 분리 이득이 없다

submitted_text IS NULL 의 의미는 "마스킹본이 만들어진 적이 없다"이며, 그것은 규칙 BLOCK 경로에서만 발생한다 (D5 · BLOCK이면 마스킹 안 함·D7 · PENDING_REVIEW 본문 보존·D14 · BLOCK⇒NULL은 필요조건). "차단됐다"는 사실은 status 가 기록하지 이 컬럼이 기록하지 않는다.

Q2. 규칙 근거와 AI 후보를 나눌 것인가 합칠 것인가 (DR-03 · 근거를 항목 단위로, DR-04 · 근거의 출처 구분)

합친다. source 컬럼으로 구분한다.

대안	기각 사유
rule_finding / ai_finding 두 테이블	SCR-02 상세 패널이 두 출처를 같은 목록 구조로 그린다. 나누면 화면이 두 번 조회하고 두 형태를 병합해야 하며, 정렬 기준도 애플리케이션으로 올라간다

합쳤을 때 생기는 위험은 "AI 후보에 규칙 ID가 붙는 것"인데, 그건 제약으로 막는다(03번 §4).

Q3. 부서×정책의 카디널리티 (DR-10 · 부서별 적용 정책)

N:M 연결 테이블 + policy.scope 이중 구조.

scope = GLOBAL → 매핑 없이 전 부서 적용 (P-PII, P-SEC)
scope = DEPT → department_policy 매핑 필요 (P-CONF, P-EMBARGO)

대안	기각 사유
policy.dept_id 단일 FK	같은 정책을 부서 수만큼 복제해야 한다. 규칙까지 복제되므로 "규칙 하나 바꾸면 N군데"가 된다
전 정책을 매핑 테이블로	전사 정책까지 매핑 행을 만들면 부서를 추가할 때 매핑 누락 = 정책 미적용이 된다. 조용히 툴린다

V3에서 홍보팀을 추가할 때 이 설계가 검증됐다 — department 한 행만 넣었는데 P-PII·P-SEC가 자동으로 적용됐고, 매핑 작업은 P-EMBARGO 2행뿐이었다.

department_policy 의 PK는 복합 (dept_id, policy_id)다. 서러게이트 PK를 두면 같은 (부서, 정책) 중복 매핑을 막지 못한다.

Q4. 시점 보존을 이력 테이블로 할 것인가 스냅샷으로 할 것인가 (DR-15 · 정책 버전 시점 보존)

policy.version + inspection.policy_snapshot 스냅샷.

대안	판단
<code>policy_audit</code> 이력 테이블	정확하지만, 정책 편집 UI가 범위 밖(0.3)이라 이력을 만들 주체가 없다 . 빈 테이블을 만드는 센
스냅샷	판정 시점에 <code>{policyId, code, version, ruleCodes[]}</code> 를 통째로 박는다. 정책이 나중에 바뀌어도 그때의 판단 근거가 남는다

정규화 원칙으로는 중복이지만, **감사 데이터의 시점 보존은 의도적 비정규화가 정답이다**. 원본은 `policy · policy_rule` 에 정규화되어 있고, JSONB는 그 시점의 사본이다. `policy_audit` 은 Future 로 남긴다(DR-22 · 확장 지점 논리 표기).

Q5. "스키마 변경 없이" AI 응답을 보관하는 타입 (DR-19 · 스키마 변경 없이 AI 응답 보관)

JSONB 3개 — `policy_snapshot`, `rule_result`, `ai_result` (+ `finding.evidence`)

대안	기각 사유
AI 응답 필드를 컬럼으로 전개	실제 LLM 교체 시 응답 스키마가 늘어난다. 그때마다 마이그레이션이 필요하면 "AI-Ready"가 거짓말이 된다
TEXT에 JSON 문자열	조회·검증이 불가능하고, 타입 있는 객체로 왕복하지 못한다

이 세 컬럼이 발표에서 **AI-Ready 원칙 "Structured Data"의 물증**이다.

6. 정규화 수준

Core 8 테이블은 **제3정규형(3NF)**을 만족한다. 의도적 예외는 아래 세 곳뿐이다.

위치	형태	정당화
<code>inspection.policy_snapshot</code>	정책·버전·규칙코드의 중복 사본	시점 보존 (DR-15 · 정책 버전 시점 보존). 원본이 바뀌어도 과거 판정이 흔들리면 안 된다
<code>inspection.rule_result</code> / <code>ai_result</code>	엔진·AI 응답 원본	스키마 무변경 확장 (DR-19 · 스키마 변경 없이 AI 응답 보관). 파생 값은 <code>inspection_finding</code> 에 정규화되어 함께 존재한다
<code>inspection_finding.code</code>	<code>policy_rule.code</code> 의 사본	AI 후보는 <code>rule_id</code> 가 NULL이라 코드를 자체 보유해야 한다. 규칙 finding에서는 중복이지만 두 출처의 컬럼 구조를 통일하는 대가다

부분 함수 종속·이행 종속은 없다. `department_policy` 의 복합 PK에 딸린 비키 속성은 `applied_at` 하나이며 PK 전체에 완전 종속한다.

7. Future Domain (Logical only) — DR-22(확장 지점 논리 표기)

DDL을 실행하지 않고 `docs/erd.dbml` 에만 표기한다. 확장 지점을 그림으로 보여주되 범위(0.3)는 지킨다.

테이블	연결	무엇을 대비하는가
<code>attachment</code>	<code>message</code> 1:N	첨부파일 메타. 텍스트 추출 후 같은 규칙 엔진 에 태운다
<code>knowledge_source</code>	AIInspector 입력	기밀 사전·NDA 목록. 현재 KEYWORD 패턴이 하는 일을 RAG 검색으로 대체
<code>policy_audit</code>	<code>policy</code> 1:N	정책 변경 이력. 현재는 Q4의 스냅샷으로 대체
<code>ai_provider_config</code>	독립	모델별 파라미터. 현재는 <code>application.yml</code> + 환경변수 (11.3)

발표 문장: "확장 지점은 상상이 아니라 ERD에 자리로 그려져 있고, 각각이 어떤 현재 구현을 대체하는지 대응이 있습니다."

8. 다음 단계로 넘기는 것

03번 스키마 설계서가 결정할 것: 1. 각 컬럼의 타입과 길이 2. R8·R9의 nullable을 약용한 잘못된 행을 **무엇으로 막을 것인가** (Q2의 미결) 3. DR-20(값 집합 DB 강제)을 CHECK로 할 것인가 ENUM 타입으로 할 것인가 4. DR-16(감사 목록 필터·정렬)의 조회 패턴에 필요한 인덱스

03. 스키마 설계서 (물리 모델)

입력: 02 ERD 설계서 출력: backend/src/main/resources/db/migration/V1__schema.sql · V3__embargo.sql 대상 DBMS: PostgreSQL 16

이 문서와 DDL이 어긋나면 DDL이 정답이다.

1. 명명 규칙

대상	규칙	예
테이블	snake_case 단수형	policy_rule, inspection_finding
PK	<테이블>_id	policy_id, finding_id
FK 컬럼	참조 테이블의 PK명 유지	policy_rule.policy_id
제약	pk_ uq_ fk_ chk_ + 대상	chk_finding_review_status
인덱스	idx_<테이블>_<컬럼들>	idx_message_user_status
enum 값	UPPER_SNAKE	PENDING_REVIEW

제약에 전부 이름을 붙인 이유: 위반 시 오류 메시지에 제약명이 그대로 나온다. chk_finding_review_status 라고 쓰면 원인을 즉시 안다. 익명 제약이면 시스템 생성 이름을 역추적해야 한다.

2. 타입 선택 기준

요구	타입	근거
PK	BIGSERIAL	감사 로그는 증가만 한다. INT 상한을 걱정하지 않는 편이 싸다
코드·enum 값	VARCHAR(n) + CHECK	\$3 참조
본문	TEXT	프롬프트 길이 상한은 애플리케이션이 관리한다(gateway.limits.max-input-chars). DB가 자르면 감사 증적이 잘린다
구조화 응답	JSONB	DR-19(스키마 변경 없이 AI 응답 보관). JSON이 아니라 JSONB — 파싱 결과를 저장하므로 조회·인덱싱이 가능하다
시각	TIMESTAMPZ	타임존 없는 TIMESTAMP 는 배포 환경(EC2 UTC)과 로컬(KST)에서 다른 값이 된다
엠바고 해제일	DATE	시각이 아니라 날짜다. TIMESTAMPZ 로 두면 "9월 20일 몇 시부터 풀리는가"라는 없는 질문이 생긴다

3. 값 집합을 CHECK로 강제한다 (DR-20 · 값 집합 DB 강제)

PostgreSQL ENUM 타입 대신 VARCHAR + CHECK 를 골랐다.

기준	ENUM 타입	VARCHAR + CHECK 
값 추가	ALTER TYPE ... ADD VALUE — 트랜잭션 안에서 쓸 수 없는 버전 제약이 있다	DROP CONSTRAINT + ADD CONSTRAINT — 일반 DDL
JPA 매핑	커스텀 타입 핸들러 필요	@Enumerated(String) 기본 지원
조회	캐스팅 필요한 경우 발생	문자열 그대로

이 선택이 V3에서 값을 했다. 엠바고 정책을 넣으며 PolicyCategory 에 EMBARGO 를 추가해야 했는데, DROP CONSTRAINT chk_policy_category → ADD CONSTRAINT ... IN ('PII', 'SECRET', 'CONFIDENTIAL', 'EMBARGO') 두 문장으로 끝났다. ENUM 타입이었으면 마이그레이션 트랜잭션 처리를 따로 고민해야 했다.

CHECK 제약 목록 (14개)

테이블	제약	허용 값
app_user	chk_app_user_role	EMPLOYEE, SECURITY_ADMIN
policy	chk_policy_category	PII, SECRET, CONFIDENTIAL, EMBARGO(V3)
policy	chk_policy_scope	GLOBAL, DEPT
policy_rule	chk_rule_type	REGEX, KEYWORD
policy_rule	chk_rule_action	MASK, BLOCK, REVIEW
policy_rule	chk_rule_severity	HIGH, MEDIUM, LOW
policy_rule	chk_rule_obligation	LEGAL, INTERNAL
policy_rule	chk_rule_mask_label	§4 참조
message	chk_message_status	ALLOWED, MASKED, BLOCKED, PENDING_REVIEW
inspection	chk_inspection_phase	INPUT, OUTPUT
inspection	chk_inspection_ai_status	SKIPPED, PENDING, COMPLETED, FAILED
inspection	chk_inspection_final_decision	ALLOW, MASK, BLOCK, PENDING
inspection	chk_inspection_decided_by	RULE, HUMAN
inspection_finding	chk_finding_source	RULE, AI
inspection_finding	chk_finding_action	MASK, BLOCK, REVIEW
inspection_finding	chk_finding_review_status	SUGGESTED, ACCEPTED, REJECTED, CONFIRMED
inspection_finding	chk_finding_ai_no_rule	§4 참조

기획서 6.2(테이블 상세)가 명시한 CHECK는 **review_status** 하나뿐이다. 나머지는 DA 판단으로 넘겼다 — 근거가 DR-20(값 집합 DB 강제)이다.

department.code에는 CHECK를 걸지 않았다. 의도적이다. 부서는 조직 변경으로 늘어나는 값이며, V3에서 홍보팀을 **INSERT** 한 줄로 추가할 수 있었던 것이 그 결과다. 정책·규칙의 값 집합은 설계가 결정하지만 부서 목록은 조직이 결정한다.

4. 업무 규칙을 제약으로 옮긴 것 (02번 Q2의 답)

ERD 단계에서 남긴 위험 — “R8·R9의 nullable을 악용한 잘못된 행” — 을 두 제약으로 막는다.

4.1 chk_rule_mask_label — DR-14(마스킹 라벨은 MASK에만)

```
CHECK (action <> 'MASK' OR mask_label IS NOT NULL)
```

근거: 기획서 7.5(충돌 해결) — “BLOCK 액션 규칙에는 mask_label이 정의되어 있지 않아 실행하면 오류가 난다.” 런타임 NPE가 될 조건을 DB에서 미리 막는다.

한 방향만 강제한다. BLOCK/REVIEW 규칙에 라벨을 다는 것은 막지 않는다 — 의미상 무해하고, 반대 방향까지 걸면 나중에 “차단 시에도 라벨을 보여주자”는 요구를 못 받는다.

4.2 chk_finding_ai_no_rule — DR-04(근거의 출처 구분)

```
CHECK (source <> 'AI' OR rule_id IS NULL)
```

근거: 기획서 6.2(테이블 상세) — “AI 후보는 NULL.” 이것이 02번 Q2에서 “합친다”고 결정한 대가를 치르는 자리다. 한 테이블에 두 출처를 담되, AI 후보가 규칙 권위를 빌리지 못하게 한다.

역방향(**RULE** 이면 **rule_id** 필수)은 걸지 않았다. 규칙 엔진이 rule_id 없이 finding을 만드는 경로를 막고 싶지 않았다. 3일 스프린트에서 병렬 작업 중인 컴포넌트를 막을 위험이 얻는 것보다 크다.

4.3 review_status DEFAULT 'SUGGESTED' — DR-05(AI 후보는 확정 전 무효)

```
review_status VARCHAR(20) NOT NULL DEFAULT 'SUGGESTED'
```

이 한 줄이 4장 책임 경계의 세 번째 강제 지점이다 (나머지 둘은 시스템 프롬프트의 금지 조항과 JSON 스키마의 **decision** 필드 부재).

애플리케이션 기본값으로만 두면 엔티티를 우회한 **INSERT** 한 번에 AI 후보가 확정 상태로 들어간다. DB 기본값이면 어떤 경로로 들어와도 SUGGESTED다.

`source='RULE'` 이면 `review_status='CONFIRMED'` 는 CHECK로 걸지 않았다. 의미상 참이지만, 규칙 엔진이 DEFAULT에 기대어 INSERT한 뒤 UPDATE하는 구현을 쓰면 INSERT가 즉시 죽는다. 대신 엔티티의 `InspectionFinding.ofRule(...)` 팩토리가 CONFIRMED를 박아서 만든다. 제약이 아니라 코드 규약으로 지키기로 한 유일한 항목이며, 검증은 07번 §3에서 데이터로 한다.

5. NOT NULL을 기획서보다 조인 것

기획서 6.2(테이블 상세) 표에 제약이 안 적힌 컬럼 중 아래를 NOT NULL로 했다.

`app_user.dept_id` · `policy.is_active` · `policy_rule.is_active` · 모든 FK(`policy_rule.policy_id` , `message.user_id` , `inspection.message_id` , `inspection_finding.inspection_id`) · 모든 `created_at` / `applied_at`

근거: nullable로 두면 `is_active IS NOT FALSE` 같은 3값 논리가 정책 로드 쿼리에 번진다. 그 쿼리가 규칙 엔진의 입구다 — 거기서 한 행이 잘못 빠지면 판정이 통째로 달라진다.

반대로 nullable을 유지한 것:

컬럼	이유
<code>message.submitted_text</code>	NULL이 "마스크링 없음"이라는 의미를 갖는다 (02번 Q1)
<code>inspection.ai_result</code>	BLOCK이면 AI를 호출하지 않는다 (7.5)
<code>inspection.final_decision</code> · <code>decided_by</code>	판정 진행 중 상태가 존재한다
<code>inspection_finding.rule_id</code> · <code>reviewed_by</code>	R8-R9. §4가 이 nullable을 조건부로 막는다
<code>policy.owner_dept_id</code>	V3에서 추가된 컬럼. 기존 행을 깨지 않으려면 nullable로 넣고 UPDATE해야 한다 (04번 §4)
<code>policy_rule.embargo_until</code>	NULL = 기한 없는 규칙. 10종 중 8종이 그렇다

6. 인덱스 — DR-16(감사 목록 필터·정렬)의 조화 패턴에서 역산

인덱스	조화 패턴	유래
<code>idx_inspection_created_at</code> (<code>created_at</code> DESC)	감사 목록의 고정 정렬	5.4 SCR-02
<code>idx_inspection_message</code> (<code>message_id</code>)	목록·상세가 항상 <code>message</code> 로 조인	8.4
<code>idx_message_user_status</code> (<code>user_id</code> , <code>status</code>)	부서 필터가 <code>user</code> 를 거치고 <code>status</code> 필터가 뒤따른다	5.4 필터
<code>idx_finding_inspection</code> (<code>inspection_id</code>)	상세의 <code>findings[]</code> , 목록의 <code>ruleCount</code>	8.4
<code>idx_policy_rule_policy</code> (<code>policy_id</code>)	정책 로드 조인	7.4-3
<code>idx_app_user_dept</code> (<code>dept_id</code>)	부서 필터 조인	5.4

복합 인덱스의 컬럼 순서: (`user_id`, `status`) 는 부서 필터가 먼저 좁히고 상태가 뒤따르는 실제 쿼리 순서를 따른다. 뒤집으면 상태만으로 필터하는 쿼리에만 유리한데, 그런 화면은 없다.

`code` 계열에는 인덱스를 따로 만들지 않았다 — PostgreSQL이 UNIQUE와 PK에 자동으로 인덱스를 만든다.

정직하게 적어 둔다: 시드 108행 규모에서는 여섯 개 전부 seq scan이 더 빠르다. 설계 의도를 남기려고 만들었다. 발표에서는 "인덱스 있습니다"가 아니라 "이 조화 패턴에 이 인덱스입니다"로 말한다.

7. 컬럼 코멘트를 남긴 이유

COMMENT ON COLUMN 을 27곳에 달았다. 예:

```
COMMENT ON COLUMN inspection_finding.review_status IS
'SUGGESTED | ACCEPTED | REJECTED | CONFIRMED. 규칙 finding은 CONFIRMED 고정 (D6 · review_status 4값)';
```

DBeaver·DataGrip·\d+ 에서 그대로 보인다. 기획서를 열지 않고도 스키마만으로 의도가 읽히는 것이 목표다 — 3일 뒤 다른 사람이 이 DB를 볼 때 문서가 옆에 있으리라는 보장이 없다.

8. 전체 스펙

컬럼 65개 × 8테이블의 타입·NULL·기본값 전수 목록은 07 검증 보고서 §2에 있다. 실제 DB에서 뽑은 것이므로 그쪽이 사실이다.

9. 다음 단계로 넘기는 것

04번 마이그레이션 설계서가 결정할 것: 1. 스키마와 시드를 한 파일로 할 것인가 나눌 것인가 2. 시드의 PK를 시퀀스에 맡길 것인가 명시할 것인가 3. 이미 적용된 마이그레이션을 수정할 수 있는 조건 4. V3(엠바고)처럼 나중에 추가되는 요구사항을 어떻게 없을 것인가

04. 마이그레이션 설계서 (Flyway)

입력: 03 스키마 설계서 출력: backend/src/main/resources/db/migration/V1__schema.sql · V2__seed.sql · V3__embargo.sql 도구: Flyway (Spring Boot 3.5.3 자동 실행, flyway-core + flyway-database-postgresql)

1. 왜 Flyway인가 — ddl-auto: update 를 쓰지 않는 이유

방식	문제
spring.jpa.hibernate.ddl-auto: update	엔티티에서 DDL을 역산한다. 컬럼 삭제·타입 변경을 하지 않고, CHECK 제약·코멘트·인덱스를 만들지 않는다. 03번 설계의 대부분이 사라진다
수동 SQL 실행	팀원 4명의 로컬 DB 상태가 각자 다르다. "내 컴퓨터에서는 되는데"의 표준 발생 경로
Flyway 	SQL이 곧 소스다. 적용 이력이 flyway_schema_history 에 남고, 빈 DB에서 항상 같은 결과가 나온다

설정은 세 줄이다.

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate    # ← 만들지 않는다. 대조만 한다
  flyway:
    enabled: true
    locations: classpath:db/migration
```

validate 가 이 설계의 안전장치다. 엔티티와 DDL이 컬럼 단위로 어긋나면 애플리케이션이 기동조차 하지 못한다. 사람이 대조표를 만들 필요가 없다 (05번 §5).

2. 파일 분할 — 03번이 넘긴 Q1

스키마와 시드를 나눈다. V1(DDL) / V2(데이터)

근거	설명
재실행 단위가 다르다	시드만 다시 넣고 싶은 일이 자주 생긴다(데모 리하설). V2는 TRUNCATE ... RESTART IDENTITY CASCADE 로 시작해 단독 재실행이 가능하다
리뷰 단위가 다르다	DDL 변경은 팀 전체에 영향이 있고, 시드 변경은 시연에만 영향이 있다
실패 원인 분리	마이그레이션이 죽었을 때 "구조 문제인가 데이터 문제인가"가 파일명으로 갈린다

V1__schema.sql	Core 8 테이블 · CHECK 17 · 인덱스 6 · 코멘트 27	(구조)
V2__seed.sql	마스터 + 감사 로그 100 + 데모 백업 3	(데이터)
V3__embargo.sql	스키마 확장 + 홍보팀 데이터	(구조 + 데이터)

V3가 돌을 겸하는 이유는 §4에 있다.

3. 시드의 PK를 명시 지정했다 — 03번이 넘긴 Q2

```
INSERT INTO department (dept_id, code, name) VALUES
(1, 'DEV', '개발팀'), ...
```

시퀀스에 맡기지 않고 PK를 SQL에 박았다.

대상	PK 명시	이유
department app_user policy policy_rule		런타임 삽입 경로가 없다(정책 편집 UI 없음, 0.3). 규칙 코드가 계약·테스트·프론트 픽스처에서 고정 ID로 참조된다
message inspection inspection_finding (V2)		데모 URL /inspections/{id} 가 걸려 있다. RESTART IDENTITY 만 일으키면 INSERT 순서 변화에 ID가 밀린다
message inspection inspection_finding (V3)	 시퀀스 채번	§5의 장에 참조

배정: 마스터 1~N / 감사 로그 1~100 / 데모 백업 101~103. message_id = inspection_id 로 맞춰 두면 디버깅 때 눈으로 따라가기 쉽다.

시퀀스 재동기화 — 빠뜨리면 첫 API 호출이 죽는다

PK를 명시하면 시퀀스는 그대로 1이다. 파일 끝에서 맞춘다.

```
SELECT setval(pg_get_serial_sequence('message','message_id'), (SELECT max(message_id) FROM message));
```

이걸 빼면 사용자의 첫 프롬프트 제출이 **duplicate key** 로 실패한다. V2 §8과 V3 §6이 그 자리다.

4. V3 — 나중에 추가된 요구사항을 었는 방법 (03번이 넘긴 Q4)

홍보팀 엠바고 정책(DR-11 · 소유 부서≠적용 부서, DR-12 · 규칙의 유효기간)은 V1·V2가 이미 적용된 뒤에 결정됐다.

4.1 V1을 고치지 않았다

상황	규칙
로컬에만 적용됨	V1을 고치고 DROP SCHEMA public CASCADE 후 재적용. 팀원에게 통보 필수
공유 DB/CI에 적용됨	V1 불변 . 변경분을 새 버전 파일로 추가

V3 시점에는 CI와 팀원 로컬에 이미 V1·V2가 적용되어 있었다. 적용된 마이그레이션을 수정하면 Flyway가 체크섬 불일치로 기동을 거부한다 — 그게 정상 동작이다.

4.2 V3가 한 일

```
-- 스키마 확장
ALTER TABLE policy ADD COLUMN owner_dept_id BIGINT; -- DR-11(소유 부서≠적용 부서)
ALTER TABLE policy_rule ADD COLUMN embargo_until DATE; -- DR-12(규칙의 유효기간)
ALTER TABLE policy DROP CONSTRAINT chk_policy_category; -- EMBARGO 추가
ALTER TABLE policy ADD CONSTRAINT chk_policy_category CHECK (category IN ('PII','SECRET','CONFIDENTIAL','EMBARGO'));

-- 데이터
INSERT INTO department (dept_id, code, name) VALUES (5, 'PR', '홍보팀');
UPDATE policy SET owner_dept_id = 4 WHERE code IN ('P-PII','P-SEC','P-CONF'); -- 기존 행 보정
INSERT INTO policy ... P-EMBARGO ...
INSERT INTO policy_rule ... EMB-NOVA-01, EMB-ATLAS-02 ...
INSERT INTO department_policy VALUES (1,4), (2,4); -- DEV, SALES
```

추가 컬럼은 **nullable**로 넣고 **UPDATE**로 채운다. **NOT NULL** 로 바로 추가하면 기존 3개 정책 행이 제약을 만족하지 못해 **ALTER** 가 실패한다. 03번 §5가 **owner_dept_id** 의 nullable을 유지한 이유가 이것이다.

4.3 V3가 구조와 데이터를 겸한 이유

§2의 분할 원칙(V1=구조 / V2=데이터)에 대한 예외다.

엠바고는 컬럼 없는 데이터가 들어갈 수 없고, 데이터 없이는 컬럼이 의미가 없다. V3__schema와 V4__seed로 나누면 둘 사이에 “컬럼은 있는데 정책이 없는” 중간 상태가 생기며, 그 상태로 배포된 서버는 홍보팀 화면이 비어 있다. 하나의 요구사항은 하나의 마이그레이션으로 끝낸다.

5. 실제로 겪은 장애와 그 수정

5.1 V3 시드가 운영 DB의 PK를 침범해 기동 실패 (2026-09-03)

증상: 이미 사용 중이던 DB에 V3를 적용하자 **duplicate key** 로 마이그레이션이 통째로 실패하고 백엔드가 기동하지 못했다.

원인: V3 초안이 홍보팀 감사 로그 5건의 **message_id** 를 104~108로 하드코딩했다. 그 DB에는 이미 사용자가 보낸 프롬프트가 104번 이후를 쓰고 있었다.

수정: **message** · **inspection** · **inspection_finding** 은 **시퀀스**에서 채번하도록 바꾸고, 행 구분은 헬퍼 테이블의 **seq** 와 **marker** 로 한다. 채번된 **message_id** 는 본문 join으로 되받는다.

```
INSERT INTO message (user_id, original_text, submitted_text, status, created_at)
SELECT 5, r.original_text, r.original_text, r.status, r.created_at FROM seed_pr_row r ORDER BY r.seq;

CREATE TABLE seed_pr_msg AS
SELECT m.message_id, r.seq, r.marker, ...
FROM seed_pr_row r JOIN message m ON m.original_text = r.original_text AND m.user_id = 5;
```

교환이 규칙이 됐다:

런타임 삽입 경로가 있는 테이블의 PK는 마이그레이션에서 하드코딩하지 않는다. V2는 빈 DB에만 적용되므로 하드코딩이 안전했지만, V3는 이미 데이터가 있는 DB에 적용된다. 같은 프로젝트 안에 서도 마이그레이션 버전마다 전제가 다르다.

department · **app_user** · **policy** · **policy_rule** 은 V3에서도 PK를 명시한다 — 런타임 삽입 경로가 없기 때문이다.

5.2 정규식 리터럴에서 백슬래시가 유실될 뻔했다

PostgreSQL은 `standard_conforming_strings = on` 이 기본이므로 일반 문자열 '...' 에서 \ 가 보존된다. E'...' 를 쓰면 \d 가 d 로 유실된다 — 규칙이 조용히 아무것도 매칭하지 않게 된다.

규칙: 마이그레이션에 E'' 리터럴을 쓰지 않는다. V2-V3에 한 개도 없다. 검증은 07번 §4의 기획서 7.2(규칙 10종) ↔ DB 바이트 대조로 한다.

5.3 random() 이 두 테이블에서 다른 값을 냈다

감사 로그 100건의 `created_at` 이 `now() - (random() * interval '7 days')` 인데, `message` 와 `inspection` 을 각각 INSERT하면 두 문장에서 `random()` 이 따로 불러 시각이 어긋난다.

해결: 파생값을 헬퍼 테이블(`seed_audit_row`)에 한 번 계산해 고정된 뒤 각 테이블이 그것을 읽는다. 헬퍼 테이블은 마이그레이션 끝에서 DROP 한다.

6. 재작업 규칙 (요약)

하고 싶은 것	방법
시드만 다시 넣기	V2를 그대로 실행. <code>TRUNCATE ... RESTART IDENTITY CASCADE</code> 로 시작하므로 재실행 가능
로컬 DB 처음부터	<code>DROP SCHEMA public CASCADE; CREATE SCHEMA public;</code> 후 <code>./gradlew bootRun</code>
적용된 마이그레이션 수정	하지 않는다. 새 버전 파일을 추가한다
규칙 하나 변경	아래 6곳을 함께 고친다

```
# 시드 단독 재적용
docker exec -i gateway-pg psql -U gateway -d gateway -v ON_ERROR_STOP=1 \
< backend/src/main/resources/db/migration/V2_seed.sql
```

규칙 하나를 바꾸면 함께 고쳐야 하는 6곳

#	위치	빠뜨리면
1	<code>V*/*.sql</code>	실서버 판정이 안 바뀐다
2	<code>frontend/dev/fixture-server.js</code> 의 RULES	픽스처 모드와 실서버가 다른 판정을 낸다
3	기획서 7.2(규칙 10종)	SSOT가 거짓이 된다
4	<code>docs/api-spec.md</code>	계약과 구현이 갈린다
5	계약 확정본	다른 컴포넌트가 옛 값을 읽는다
6	테스트 기대값	CI가 초록인데 동작이 틀린다

7. 검증 — 이 설계가 실제로 재현되는가

빈 스키마에서 Flyway가 V1→V2→V3를 처음부터 적용하는 것을 확인했다.

installed_rank	version	description	success	소요
1	1	schema	t	44ms
2	2	seed	t	270ms
3	3	embargo	t	20ms

이어서 `ddl-auto: validate` 통과 + 테스트 41건 전부 통과. 상세는 [07 검증 보고서](#).

8. 다음 단계로 넘기는 것

05번 엔티티 매핑 설계서가 결정할 것: 1. CHECK로 강제한 값 집합을 Java에서 어떻게 표현할 것인가 2. JSONB 3개를 문자열로 다룰 것인가 타입 있는 객체로 다룰 것인가 3. 복합 PK(`department_policy`)를 어떻게 매핑할 것인가 4. DR-16(감사 목록 필터·정렬)의 선택 필터(부서·상태·기간, 각각 생략 가능)를 어떤 쿼리 방식으로 만들 것인가

05. 엔티티·리포지토리 매핑 설계서

입력: 04 마이그레이션 설계서 출력: backend/src/main/java/com/skala/gateway/domain/** 스택: Spring Boot 3.5.3 · Spring Data JPA · Hibernate 6 · Java 21

이 단계는 DDL이 먼저 있고 엔티티가 그것을 따라간다. 반대가 아니다 — ddl-auto: validate 가 그 방향을 강제한다(\$5).

1. 산출물

종류	개수	위치
엔티티	8 + 복합 PK 클래스 1	domain/*.java
enum	14	domain/enums/*.java
JSONB 매핑 타입	2 (+ AI 응답 record)	domain/jsonb/*.java
Repository	8 + Specification 1	domain/repository/*.java

2. 값 집합을 Java enum으로 (04번이 넘긴 Q1)

CHECK 제약의 값 목록을 그대로 Java enum 14개로 옮기고 @Enumerated(EnumType.STRING) 으로 매핑한다.

```
@Enumerated(EnumType.STRING)
@Column(name = "review_status", nullable = false, length = 20)
private ReviewStatus reviewStatus;
```

EnumType.ORDINAL 을 쓰지 않는다. 서수로 저장하면 enum 상수 순서를 바꾸는 순간 기존 데이터의 의미가 통째로 뒤집힌다. 감사 데이터에서 이런 복구 불가능한 오염이다. 문자열로 저장하면 DB만 봐도 값이 읽히고, CHECK 제약과 같은 문자열로 대조된다.

enum ↔ CHECK 대조 (14종 전부 일치)

enum	값	대응 CHECK
UserRole	EMPLOYEE, SECURITY_ADMIN	chk_app_user_role
PolicyCategory	PII, SECRET, CONFIDENTIAL, EMBARGO	chk_policy_category
PolicyScope	GLOBAL, DEPT	chk_policy_scope
RuleType	REGEX, KEYWORD	chk_rule_type
RuleAction	MASK, BLOCK, REVIEW	chk_rule_action
Severity	HIGH, MEDIUM, LOW	chk_rule_severity
Obligation	LEGAL, INTERNAL	chk_rule_obligation
MessageStatus	ALLOWED, MASKED, BLOCKED, PENDING_REVIEW	chk_message_status
InspectionPhase	INPUT, OUTPUT	chk_inspection_phase
AiStatus	SKIPPED, PENDING, COMPLETED, FAILED	chk_inspection_ai_status
FinalDecision	ALLOW, MASK, BLOCK, PENDING	chk_inspection_final_decision
DecidedBy	RULE, HUMAN	chk_inspection_decided_by
FindingSource	RULE, AI	chk_finding_source
ReviewStatus	SUGGESTED, ACCEPTED, REJECTED, CONFIRMED	chk_finding_review_status

이중 방어의 구조: Java enum이 애플리케이션 경로를 막고, CHECK가 그 외 모든 경로를 막는다. 어느 한쪽만 있으면 뚫린다. 07번 \$5가 실제로 뚫리지 않는 것을 확인한다.

3. JSONB를 타입 있는 객체로 (04번이 넘긴 Q2)

```
@JdbcTypeCode(SqlTypes.JSON)
@Column(name = "policy_snapshot", nullable = false, columnDefinition = "jsonb")
private PolicySnapshot policySnapshot;
```

Hibernate 6의 `@JdbcTypeCode(SqlTypes.JSON)` 으로 `record` 와 직접 왕복한다.

```
public record RuleResult(List<RuleMatch> matches, List<String> appliedRuleCodes) { ... }
public record PolicySnapshot(List<PolicyRef> policies) { ... }
```

대안	기각 사유
<code>String</code> 필드 + 수동 <code>ObjectMapper</code>	서비스 계층마다 파싱·직렬화 코드가 흩어진다. 오타가 런타임에야 드러난다
<code>Map<String, Object></code>	필드명이 문자열 키라 컴파일러가 아무것도 잡아주지 못한다

이것이 발표에서 AI-Ready "Structured Data" 원칙의 물증이다. AI 응답 스키마가 늘어나면 `record` 에 필드를 추가할 뿐 마이그레이션이 없다.

```
policy_snapshot → PolicySnapshot[policies=[PolicyRef[policyId=1, code=P-PII, version=3, ruleCodes=[...]]]
rule_result     → RuleResult[matches=[RuleMatch[...], span=[18,26], matchedKeyword=null, ...], appliedRuleCodes=[...]]
ai_result       → AiAssessment[riskCandidates=[RiskCandidate[...], evidence=[Evidence[...]]], reviewRequired=true]
```

`RuleMatch.embargoUntil` 은 `LocalDate` 가 아니라 `String` 이다. 이 `record` 는 API 응답 JSON에 그대로 실리므로, 낱짜 형식이 Jackson 설정에 좌우되면 계약이 흔들린다.

4. 복합 PK 매핑 (04번이 넘긴 Q3)

`department_policy` 는 (`dept_id`, `policy_id`) 복합 PK다.

```
@Embeddable
public class DepartmentPolicyId implements Serializable {
    @Column(name = "dept_id") private Long deptId;
    @Column(name = "policy_id") private Long policyId;
}

@Entity @Table(name = "department_policy")
public class DepartmentPolicy {
    @EmbeddedId private DepartmentPolicyId id;
    ...
}
```

대안	기각 사유
<code>@IdClass</code>	필드를 엔티티와 ID 클래스 양쪽에 중복 선언해야 한다
<code>@ManyToMany</code> 로 매핑 테이블을 감추기	<code>applied_at</code> 컬럼을 가질 수 없고, 매핑 존재 여부를 직접 조회하지 못한다. 정책 로드 쿼리가 <code>exists (...)</code> 로 이 테이블을 직접 본다

5. `ddl-auto: validate` — 사람이 대조표를 만들지 않는다

```
spring.jpa.hibernate.ddl-auto: validate
```

기동 시 Hibernate가 엔티티 65개 필드와 DDL 65개 컬럼을 이름·타입 단위로 대조한다. 어긋나면 애플리케이션이 뜨지 않는다.

발표에서 쓸 문장: "엔티티와 DDL이 일치한다는 것을 문서로 주장하지 않습니다. 일치하지 않으면 서버가 기동하지 않습니다."

`update` 나 `create` 를 쓰지 않는 이유는 04번 §1에 있다 — 그쪽은 엔티티에서 DDL을 역산하므로 CHECK 제약·코멘트·인덱스가 전부 사라진다.

6. Repository 설계

6.1 정책 로드 — 규칙 엔진의 입구

`PolicyRuleRepository.findActiveByDept(deptId)` 하나가 기획서 7.3(부서 매트릭스) 매트릭스 전체를 대신한다.

```
where r.isActive = true and p.isActive = true
and (p.scope = GLOBAL
    or exists (select 1 from DepartmentPolicy dp
        where dp.id.policyId = p.policyId and dp.id.deptId = :deptId))
order by <REGEX 먼저>, <severity 내림차순>, r.code
```

정렬이 곧 실행 순서다. 기획서 7.4(관정 절차)-4-7.4-5가 "REGEX를 severity 내림차순으로 먼저, 그 다음 KEYWORD"라고 정했고, 동렬은 rule code 사전순(7.6의 라벨 충돌 해결과 같은 기준)이다. 쿼리에 서 순서를 보장하면 엔진이 정렬을 다시 하지 않는다.

이 시그니처는 DA → 규칙 엔진 인계 계약이므로 바꾸지 않는다.

6.2 감사 목록의 선택 필터 (04번이 넘긴 Q4) — 겪은 문제와 해결

처음 시도(실패): JPQL의 (:param is null or ...) 관용구

```
and (:from is null or i.createdAt >= :from)
```

```
ERROR: could not determine data type of parameter $5
```

OffsetDateTime null을 바인딩하면 JDBC 드라이버가 타입 OID를 보내지 못하고, PostgreSQL이 \$5 IS NULL 의 타입을 결정하지 못한다. deptId (bigint)·status (varchar)는 통과하는데 타임 스탬프만 죽어서 더 헛갈렸다.

해결: JpaSpecificationExecutor + InspectionSpecs

```
public static Specification<Inspection> of(Long deptId, MessageStatus status,
    OffsetDateTime from, OffsetDateTime to) {
    // null인 필터는 Specification 자체를 만들지 않는다 → SQL에 나가지 않는다
}
```

"값이 없으면 전체"가 조건 부재로 자연스럽게 성립한다. null 파라미터가 애초에 SQL에 도달하지 않는다.

경계 처리: to 는 미만 (<)이다. 포함하면 하루 단위 필터가 다음 날 00:00을 삼킨다.

6.3 N+1 방지

목록 조회에 @EntityGraph 를 붙였다.

```
@EntityGraph(attributePaths = {"message", "message.user", "message.user.department"})
```

없으면 20행 목록에서 부서·사용자 조회가 행마다 나간다(1 + 20×3 쿼리). 감사 목록은 deptId 필터와 부서명 표시 때문에 항상 이 경로를 탄다.

7. 팩토리 메서드로 지키는 규약

03번 §4.3에서 CHECK로 걸지 않기로 한 규칙이 하나 있다 — "규칙 finding의 review_status 는 CONFIRMED 고정"(D6 · review_status 4값).

제약 대신 엔티티 팩토리가 강제한다.

```
InspectionFinding.ofRule(inspection, rule, category, spanStart, spanEnd) // → CONFIRMED 고정
InspectionFinding.ofAi(inspection, code, category, rationale, evidence) // → SUGGESTED (DB DEFAULT)
```

setter로 임의 조립하는 경로를 열어두지 않았다. 제약이 아닌 규약이므로 검증이 필요하고, 그 검증을 07 검증 보고서 §3이 데이터로 수행한다.

8. 매핑 요약

엔티티	테이블	특이사항
Department	department	—
AppUser	app_user	@ManyToOne → Department
Policy	policy	ownerDept @ManyToOne (nullable)
PolicyRule	policy_rule	embargoUntil LocalDate
DepartmentPolicy	department_policy	@EmbeddedId
Message	message	submittedText nullable — 의미를 갖는 NULL
Inspection	inspection	JSONB 3
InspectionFinding	inspection_finding	JSONB 1, nullable FK 2, 팩토리 2

모든 @ManyToOne 은 FetchType.LAZY 다. 기본값 EAGER는 목록 조회에서 필요 없는 조인을 끌고 온다.

9. 다음 단계로 넘기는 것

06번 추적 매트릭스가 확인할 것: DR-01~DR-22가 전부 어딘가에 도달했는가. 07번 검증 보고서가 확인할 것: 그 도달이 실제 DB에서 참인가.

06. 요구사항 추적 매트릭스 (RTM)

목적: 데이터 요구사항 하나가 ERD → 스키마 → 마이그레이션 → 엔티티 → API/화면까지 끊기지 않고 도달했는지 한 표로 확인한다.

읽는 법: 어느 칸이든 비면 그 요구사항은 설계만 되고 구현되지 않았다는 뜻이다. Q&A에서 "그건 어디에 구현되어 있나요"에 이 표 한 줄로 답한다.

1. 기능 요구사항

DR	요구사항	테이블·컬럼	마이그레이션	엔티티/코드	API·화면	검증
DR-01	원문/전송본 분리	message.original_text message.submitted_text	V1	Message	POST /messages 응답 submittedText · SCR-01	07 §3 불변식 1~3
DR-02	검사 1회 = 1레코드	inspection	V1	Inspection	GET /inspections/{id} · SCR-02	07 §2
DR-03	근거를 항목 단위로	inspection_finding	V1	InspectionFinding	응답 findings[] · SCR-02 상세	07 §2
DR-04	근거의 출처 구분	inspection_finding.source chk_finding_ai_no_rule	V1	FindingSource	findings[].source	07 §3 불변식 5~7
DR-05	AI 후보는 확정 전 무효	review_status DEFAULT 'SUGGESTED' chk_finding_review_status	V1	ReviewStatus InspectionFinding.ofAi	PATCH .../findings/{id} · UC-06(AI 제안 확정)	07 §5 테스트 1~2
DR-06	사람의 확정 기록	reviewed_by reviewed_at inspection.decided_by	V1	DecidedBy	PATCH 응답 reviewedBy	07 §3 불변식 2
DR-07	비동기 AI 상태 추적	inspection.ai_status completed_at	V1	AiStatus	202 + 폴링 (8.5) · UC-03-04(AI 판정 요청과 폴링)	테스트 DemoCase
DR-08	상태 ↔ 판정 무모순	message.status inspection.final_decision	V1	MessageStatus FinalDecision	7.5 충돌 해결	07 §3 불변식 7
DR-09	기준은 코드가 아니라 데이터	policy_rule.pattern action severity	V1 · V3	PolicyRule	GET /policies · SCR-01 정책 패 널	07 §4
DR-10	부서별 적용 정책	department_policy (복합 PK) policy.scope	V1 · V2 · V3	DepartmentPolicy PolicyRuleRepository.findActiveByDept	GET /policies?deptId= · UC-07(부서 정책 조회)	07 §4 매트릭스
DR-11	소유 부서 ≠ 적용 부서	policy.owner_dept_id	V3	Policy.ownerDept	7.1 표	07 §4 정책 표
DR-12	규칙의 유효기간	policy_rule.embargo_until	V3	PolicyRule.embargoUntil RuleEngine 만료 제외	PolicyRuleDto.embargoUntil	테스트 Embargo 건
DR-13	규칙 코드가 공통 식별자	policy_rule.code UNIQUE	V1	PolicyRule.code	findings[].code · 발표 대사	07 §4 바이트 대조
DR-14	마스킹 라벨은 MASK에만	chk_rule_mask_label	V1	Masker	7.6	07 §3 불변식 6~7
DR-15	판정 시점의 정책 버전 보존	policy.version inspection.policy_snapshot	V1 · V2	PolicySnapshot record	응답 policyVersion	07 §2 JSONB
DR-16	검사 목록 필터·정렬	인덱스 6개	V1	InspectionSpecs @EntityGraph	GET /inspections?... · SCR-02	07 §2 인덱스
DR-17	span은 원문 기준	span_start span_end	V1	RuleMatch.span	SCR-01 하이라이트	테스트 RuleEngineD
DR-18	로그인 없는 사용자 식별	app_user · app_user.dept_id NOT NULL	V1 · V2	CurrentUserId	X-User-Id 헤더 (8.1)	테스트 MasterI

2. 비기능 요구사항

DR	요구사항	구현 위치	마이그레이션	검증
DR-19	AI 응답을 스키마 변경 없이 보관	<code>policy_snapshot</code> <code>rule_result</code> <code>ai_result</code> <code>evidence</code> (JSONB 4)	V1	07 §2 — V3에서 AI 관련 컬럼 추가 0개
DR-20	값 집합을 DB 수준에서 강제	CHECK 17개 ↔ Java enum 14개	V1 · V3	07 §5 부정 테스트 6건
DR-21	시드에 개인정보 형태 문자열 없음	V2 템플릿 20종 · 라벨 적용본 저장	V2 · V3	07 §6
DR-22	확장 지점을 논리 모델로 표기	<code>docs/erd.dbml</code> Future 4 <code>inspection.phase= 'OUTPUT'</code> 자리	— (DDL 없음)	07 §2 — 실제 테이블 8개뿐

3. 역방향 추적 — 테이블이 어떤 요구사항에서 나왔는가

“이 테이블 왜 있나요”에 대한 답이다. 근거 없는 테이블이 하나도 없다.

테이블	유래 DR	없으면 무엇이 불가능한가
<code>department</code>	DR-10(부서별 적용 정책), DR-18(로그인 없는 사용자 식별)	부서별 정책 차등 적용 자체가 불가능
<code>app_user</code>	DR-18(로그인 없는 사용자 식별), DR-06(사람의 확정 기록)	제출자·확정자를 식별할 수 없다
<code>policy</code>	DR-09(기준은 코드가 아니라 데이터), DR-11(소유 부서≠적용 부서), DR-15(정책 버전 시점 보존)	규칙을 묶는 단위와 버전 축이 사라진다
<code>policy_rule</code>	DR-09(기준은 코드가 아니라 데이터), DR-12(규칙의 유효기간), DR-13(규칙 코드가 공통 식별자), DR-14(마스킹 라벨은 MASK에 만)	판단 기준이 코드로 되돌아간다
<code>department_policy</code>	DR-10(부서별 적용 정책)	데모 Case B/C(부서로 갈리는 판정)가 성립하지 않는다
<code>message</code>	DR-01(원문/전송본 분리), DR-18(로그인 없는 사용자 식별)	감사 증적의 원문이 남지 않는다
<code>inspection</code>	DR-02(검사 1회=1레코드), DR-07(비동기 AI 상태 추적), DR-15(정책 버전 시점 보존), DR-19(스키마 변경 없이 AI 응답 보관)	비동기 상태와 시점 스냅샷을 둘 곳이 없다
<code>inspection_finding</code>	DR-03(근거를 항목 단위로)~06, DR-17(span은 원문 기준)	“왜 차단됐는가”를 설명할 수 없다

4. 화면·API 역방향 — 무엇이 무엇을 읽는가

화면/API	읽는 테이블	핵심 컬럼
SCR-01 정책 패널 · GET /policies?deptId=	<code>policy</code> <code>policy_rule</code> <code>department_policy</code>	<code>scope</code> , <code>embargo_until</code>
SCR-01 제출 · POST /messages	<code>policy_rule</code> (읽기) → <code>message</code> <code>inspection</code> <code>inspection_finding</code> (쓰기)	<code>pattern</code> , <code>action</code> , <code>submitted_text</code>
SCR-01 풀링 · GET /inspections/{id}	<code>inspection</code> <code>inspection_finding</code>	<code>ai_status</code> , <code>findings[]</code>
SCR-02 목록 · GET /inspections?...	<code>inspection</code> ✕ <code>message</code> ✕ <code>app_user</code> ✕ <code>department</code>	인덱스 6개가 이 조인을 받는다
SCR-02 확정 · PATCH .../findings/{id}	<code>inspection_finding</code>	<code>review_status</code> , <code>reviewed_by</code> , <code>reviewed_at</code>
계정 전환 · GET /users?deptId=	<code>app_user</code> <code>department</code>	X-User-Id 의 원천

5. 커버리지

구분	개수	상태
데이터 요구사항	22	전부 스키마에 도달
Core 테이블	8	전부 DR 유래 있음
CHECK 제약	17	DR-20(값 집합 DB 강제) · DR-14(마스킹 라벨은 MASK에만) · DR-04(근거의 출처 구분) 유래
인덱스	6	DR-16(감사 목록 필터·정렬) 유래
Future 테이블	4	DR-22(확장 지점 논리 표기) — 의도적 미구현 (Logical only)
미구현 DR	0	—

"설계는 했는데 구현 안 된 것"이 없다는 사실 자체가 이 표의 결론이다.

07. 스키마 검증 보고서

검증일: 2026-09-03 대상: V1__schema.sql · V2__seed.sql · V3__embargo.sql 환경: PostgreSQL 16.14 (Docker, 빈 DB에서 처음부터 적용) · Spring Boot 3.5.3 · Java 21 질문: 지금 스키마가 이 요구사항 정의를 실제로 반영하고 있는가?

0. 결론

구분	결과
Flyway V1→V2→V3 빈 DB 적용	✅ 3건 전부 성공
ddl-auto: validate (엔티티 65필드 ↔ DDL 65컬럼)	✅ 통과
테스트 스위트	✅ 41건 전부 통과, 실패 0
기획서 6.2(테이블 상세) 컬럼 전수 대조	✅ 65/65 타입·길이 일치
기획서 7.1(정책 4종) 정책 4종	✅ 카테고리·scope·소유·적용 전부 일치
기획서 7.2(규칙 10종) 규칙 10종	✅ 패턴 바이트 단위 일치
기획서 7.3(부서 매트릭스) 부서 매트릭스	✅ 실데이터 파생 결과가 표와 동일
업무 불변식 8종	✅ 위반 0건
계약 부정 테스트 6종	✅ 전부 정상 거부
발견 사항	⚠️ 5건 (치명 0 / 개선 2 / 문서 동기화 2 / 정보 1)

요구사항 DR-01~DR-22는 전부 스키마에 반영되어 있다. 발견 사항 5건은 모두 "반영은 됐으나 강제가 코드 규약에 의존" 또는 "문서가 최신이 아님"이며, 판정 결과를 바꾸지 않는다.

1. 재현 절차

```
docker run -d --name gateway-pg-verify -e POSTGRES_DB=gateway -e POSTGRES_USER=gateway \
-e POSTGRES_PASSWORD=gateway -p 55433:5432 postgres:16-alpine

# 빈 스키마에서 Flyway가 처음부터 적용하는지 확인
docker exec gateway-pg-verify psql -U gateway -d gateway -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
cd backend && DB_URL=jdbc:postgresql://localhost:55433/gateway ./gradlew test
```

installed_rank	version	description	success	소요
1	1	schema	t	44ms
2	2	seed	t	270ms
3	3	embargo	t	20ms

Flyway가 적용했다는 것은 ddl-auto: validate 도 통과했다는 뜻이다 — validate가 실패하면 Spring 컨텍스트가 뜨지 않아 테스트가 한 건도 실행되지 않는다.

테스트 클래스	건수
MasterDataApiTest	10
ReviewApiTest	9
DemoCaseApiTest	6
RuleEngineDemoCaseTest	6
EmbargoRuleTest	4
MaskerTest	4
InspectionAiResultSinkTest	2
합계	41 / 실패 0

2. 구조 대조

2.1 테이블

실제 생성된 BASE TABLE 8개. Future 4개는 없다 → DR-22(확장 지점 논리 표기) 준수 (설계는 docs/erd.dbml에만).

```
app_user · department · department_policy · inspection
inspection_finding · message · policy · policy_rule
```

2.2 컬럼 전수 대조

컬럼 65개의 타입·길이·NULL 허용·기본값을 기획서 6.2(테이블 상세) 표와 대조했다. 전부 일치.

테이블	컬럼 수	결과
department	3	✓
app_user	6	✓
policy	9 (V3의 owner_dept_id 포함)	✓
policy_rule	13 (V3의 embargo_until 포함)	✓
department_policy	3	✓
message	6	✓
inspection	11	✓
inspection_finding	14	✓

기본값 3개도 의도대로다: inspection.phase='INPUT' · inspection_finding.review_status='SUGGESTED' · policy.version=1.

2.3 제약·인덱스 인벤토리

종류	개수	비고
PRIMARY KEY	8	department_policy 는 복합 PK
UNIQUE	4	department.code app_user.email policy.code policy_rule.code
FOREIGN KEY	10	V3의 fk_policy_owner_dept 포함
CHECK	17	03 53 목록과 일치
INDEX (명시)	6	DR-16(감사 목록 필터·정렬)의 조회 패턴

2.4 JSONB — DR-19(스키마 변경 없이 AI 응답 보관) 검증

policy_snapshot · rule_result · ai_result · inspection_finding.evidence 4개.

V3가 AI 관련 컬럼을 0개 추가했다. 엠바고라는 새 정책 축이 들어오면서 policy_rule.embargo_until 이 rule_result.matches[].embargoUntil 로 흘러갔는데, JSONB 안에서만 늘어났고 inspection 테이블 구조는 그대로다. “시 스키마가 확장해도 마이그레이션이 없다”는 주장이 실제 변경 이력으로 증명된 자리다.

3. 업무 불변식 (실데이터 108건)

#	불변식	근거	위반
1	규칙 판정 BLOCKED ⇒ submitted_text IS NULL	D5(BLOCK이면 마스크 안 함)·D7(PENDING_REVIEW 본문 보존)	0
2	사람 확정 BLOCKED는 본문 보존	D14(BLOCK⇒NULL은 필요조건)	0 (해당 1건 본문 있음)
3	PENDING_REVIEW는 본문 보존	D7(PENDING_REVIEW 본문 보존)	0
4	RULE finding은 review_status='CONFIRMED'	D6(review_status 4값)	0
5	AI finding은 rule_id IS NULL	6.2	0
6	MASK 규칙은 mask_label 필수	D5(BLOCK이면 마스크 안 함)	0
7	message.status ↔ inspection.final_decision 정합	7.5	0
8	고아 inspection (참조 message 없음)	FK	0

불변식 1~2가 함께 통과한 것이 중요하다. BLOCKED ⇒ NULL 은 거짓이고 decided_by='RULE' AND BLOCKED ⇒ NULL 이 참이다(D14 · BLOCK⇒NULL은 필요조건). 앞의 명제로 테스트를 짜면 통과하지 못한다.

불변식 4는 CHECK가 아니라 코드 규약이 지킨다(03 §4.3, 05 §7). 제약이 없는 항목이므로 이 데이터 검증이 유일한 확인 수단이다.

데이터 분포

상태	V2 감사 100	V2 백업 3	V3 홍보팀 5	합계
ALLOWED	55	1 (Case C)	3	59
MASKED	25	0	2	27
BLOCKED	12	2 (Case A-B)	0	14
PENDING_REVIEW	8	0	0	8
				108

부서 분포 DEV 47 / SALES 36 / HR 20 / PR 5 = 108. 기획서 10.5(감사 로그 시드) 계획과 일치.

4. 정책·규칙 대조 — DR-09 ~ DR-13

4.1 정책 4종 (기획서 7.1(정책 4종))

id	code	category	scope	소유	적용	규칙
1	P-PII	PII	GLOBAL	INFOSEC	전사	4
2	P-SEC	SECRET	GLOBAL	INFOSEC	전사	3
3	P-CONF	CONFIDENTIAL	DEPT	INFOSEC	HR, SALES	1
4	P-EMBARGO	EMBARGO	DEPT	PR	DEV, SALES	2

✅ 기획서 7.1(정책 4종)과 완전 일치. DR-11(소유 ≠ 적용)이 4행에서 실제로 같린다.

4.2 규칙 10종 — 패턴 바이트 대조

기획서 7.2(규칙 10종) 마크다운 표의 패턴 열을 파싱(\ | → | 복원)해 DB 값과 바이트 단위 비교.

✓ PII-RRN-01 (17자)	✓ SEC-AWSKEY-01 (16자)
✓ PII-CARD-02 (21자)	✓ SEC-DBURL-02 (36자)
✓ PII-PHONE-03 (26자)	✓ SEC-PRIVIP-03 (68자)
✓ PII-EMAIL-04 (23자)	✓ CONF-CLIENT-01 (21자)
✓ EMB-NOVA-01 (18자)	✓ EMB-ATLAS-02 (10자)

기획서 10행 / DB 10행 — type · pattern · action · mask_label · severity · obligation · embargo_until 전 항목 일치

백슬래시가 살아 있다 — 04 §5.2의 E* 금지 규칙이 지켜졌다는 뜻이다. \d 가 d 로 유실됐다면 규칙이 조용히 아무것도 매칭하지 않는다.

4.3 부서 매트릭스 — 실데이터에서 역산 (기획서 7.3(부서 매트릭스))

표를 읽은 것이 아니라 policy · department_policy · policy.scope 로 쿼리해서 만든 결과다.

부서	적용 정책
DEV	P-PII(전사) P-SEC(전사) P-EMBARGO(매핑)
SALES	P-PII(전사) P-SEC(전사) P-CONF(매핑) P-EMBARGO(매핑)
HR	P-PII(전사) P-SEC(전사) P-CONF(매핑)
INFOSEC	P-PII(전사) P-SEC(전사)
PR	P-PII(전사) P-SEC(전사)

✅ 기획서 7.3(부서 매트릭스) 매트릭스와 완전 일치.

여기가 DR-10(부서별 적용 정책)의 증명이다. DEV에 P-CONF가 없어서 데모 Case C가 ALLOW로 갈리고, PR에 P-EMBARGO가 없어서 "발표 주체는 자기 엠바고에 걸리지 않는다"가 성립한다. 표를 옮겨 적은 것이 아니라 데이터가 그렇게 나온다.

5. 제약 부정 테스트 — DR-20(값 집합 DB 강제)이 실제로 막는가

전부 트랜잭션 안에서 실행 후 롤백했다.

#	시도	기대	실제
1	<code>review_status</code> 미지정 INSERT	SUGGESTED 저장	✅ SUGGESTED — DR-05(AI 후보는 확정 전 무효)가 DB 수준에서 강제됨
2	<code>review_status='APPROVED'</code>	거부	✅ <code>chk_finding_review_status</code> 위반
3	<code>source='AI' + rule_id=1</code>	거부	✅ <code>chk_finding_ai_no_rule</code> 위반
4	<code>action='MASK' + mask_label</code> 없음	거부	✅ <code>chk_rule_mask_label</code> 위반
5	<code>policy.category='OTHER'</code>	거부	✅ <code>chk_policy_category</code> 위반
6	<code>app_user.dept_id=999</code>	거부	✅ <code>fk_app_user_dept</code> 위반
7	<code>department_policy</code> 중복 매핑	거부	✅ <code>pk_department_policy</code> 위반

테스트 1이 4장 책임 경계의 물증이다. 엔티티를 우회한 직접 INSERT인데도 SUGGESTED 가 박힌다 — "AI는 결정하지 못한다"가 코드가 아니라 스키마의 성질이다.

6. 시드 안전성 — DR-21(시드에 개인정보 없음)

- 감사 로그 100건의 `original_text` 는 처음부터 라벨 적용본이다 (... (참고 정보: [전화번호]))
- 템플릿 20종 전부 평범한 업무 프롬프트
- 데모 백업 Case A도 라벨 적용본으로 저장 (기획서 10.4(데모 케이스)의 원문 문자열을 그대로 넣지 않음)
- `matchedKeyword` 는 KEYWORD 매칭에만 값이 있다 — REGEX 매칭 문자열을 넣으면 주만번호 원문이 `rule_result` JSONB에 남는다
- 이름은 김00 형식, 이메일은 example.com

✅ 실제 개인정보 형태 문자열 0건.

7. 발견 사항

치명적 결함은 없다. 아래 5건은 개선·동기화 항목이다.

⚠️ F-01 (개선) — `inspection` 에 (`message_id`, `phase`) 유일 제약이 없다

기획서 6.3(관계 정의): "message — inspection 1:N, phase별 1건"

실측: 같은 `message_id` 에 `phase='INPUT'` inspection을 2건 삽입했더니 성공했다.

```
insert into inspection (message_id, phase, ...) values (1, 'INPUT', ...); -- 통과
select count(*) from inspection where message_id=1 and phase='INPUT';      -- 2
```

영향: 현재 데이터는 위반 0건이고, `InspectionService` 가 message당 1건만 만든다. 다만 재시도 로직이나 동시 요청이 들어오면 감사 목록에 같은 프롬프트가 두 줄로 뜬다. "phase별 1건"이 설계 의도인데 스키마가 강제하지 않는 유일한 관계 제약이다.

제안 (V4 후보):

```
ALTER TABLE inspection ADD CONSTRAINT uq_inspection_message_phase UNIQUE (message_id, phase);
```

현재 데이터가 위반 0건이므로 무중단으로 추가 가능하다.

⚠️ F-02 (개선) — `inspection_finding.category` 에만 CHECK가 없다

enum 성격의 컬럼 중 이것 하나만 CHECK가 없다.

실측: `category='NOT_A_CATEGORY'` 삽입이 성공했다.

영향이 단순한 누락보다 크다. 엔티티는 이 컬럼을 `PolicyCategory` enum으로 매핑한다 (05 §2). 즉 쓰기는 되는데 읽기가 터진다 — 잘못된 값이 한 번 들어가면 그 행을 포함하는 감사 목록 조회가 통째로 실패한다. DB가 조용히 받아주고 애플리케이션이 나중에 죽는, DR-20(값 집합 DB 강제)이 막으려던 바로 그 형태다.

제안 (V4 후보):

```
ALTER TABLE inspection_finding
ADD CONSTRAINT chk_finding_category CHECK (category IN ('PII','SECRET','CONFIDENTIAL','EMBARGO'));
```

현재 데이터는 전부 유효값이라 무중단으로 추가 가능하다.

 F-03 (문서 동기화) — `submitted_text` 설명이 D14(BLOCK⇒NULL은 필요조건) 이전 문구다

위치	현재 문구
<code>V1__schema.sql:117</code>	NULL이면 미전송(BLOCK 판정)
<code>docs/erd.dbml:84</code>	NULL이면 미전송(BLOCK)
기획서 6.2(테이블 상세) (정답)	NULL은 **마스킹본이 생성된 적이 없음**. 규칙 BLOCK 경로에서만 발생. PENDING_REVIEW와 사람이 확인한 BLOCK은 본문을 보존 (D7 · PENDING_REVIEW 본문을 보존)

왜 중요인가: §3 불변식 1-2가 정확히 이 지점이다. 주석을 그대로 믿고 `BLOCKED ⇒ submitted_text IS NULL` 로 테스트를 짜면 **사람 확정 BLOCK 1건 때문에 실패한다**. DDL 주석은 스키마만 보는 사람에게 유일한 설명이라 오해 비용이 크다.

제안: 두 곳의 문구를 기획서 6.2(테이블 상세)에 맞춘다. `COMMENT ON COLUMN` (131행)은 '마스킹 적용 후 본문. NULL이면 미전송' 으로 중립적이라 그대로 뒀도 무방하다.

 F-04 (문서 동기화) — `docs/erd.dbml` 의 매핑 행 수가 V3 이전 값이다

<code>docs/erd.dbml:77</code>	Note: '... 시드는 (SALES,P-CONF), (HR,P-CONF) 2행뿐'
-------------------------------	---

실측: V3 이후 `department_policy` 는 4행이다 — (SALES,P-CONF) (HR,P-CONF) (DEV,P-EMBARGO) (SALES,P-EMBARGO).

발표 자료용 ERD 이미지를 이 파일에서 export하므로, 그림에 옛 숫자가 실린다.

제안: '시드 4행 — P-CONF 2(SALES,HR), P-EMBARGO 2(DEV,SALES)' 로 갱신.

 F-05 (정보) — V3 추가 컬럼의 물리 순서가 테이블 끝이다

`policy.owner_dept_id` 와 `policy_rule.embargo_until` 은 `ALTER TABLE ADD COLUMN` 으로 들어가 **테이블의 마지막 컬럼**이다. 기획서 6.2(테이블 상세) 표는 각각 `scope` 뒤, `description` 뒤에 그린다.

동작에 영향 없다. 모든 쿼리가 컬럼명을 명시하고, `SELECT *` 의존 코드가 없다. `\d policy` 출력 순서가 기획서 표와 다르다는 것만 알고 있으면 된다. 순서를 맞추려면 테이블 재생성이 필요한데, 04 §4.1의 "적용된 마이그레이션은 고치지 않는다" 규칙에 비해 얻는 것이 없다. 수정하지 않는다.

8. 의도적으로 두는 것 (발견 사항 아님)

혼동을 막기 위해 명시한다. 아래는 누락이 아니라 결정이다.

항목	결정 근거
<code>department.code</code> 에 CHECK 없음	부서는 조직이 늘리는 값이다. V3에서 홍보팀을 <code>INSERT</code> 한 줄로 추가할 수 있었던 이유 (03 §3)
<code>source='RULE' ⇒ CONFIRMED</code> CHECK 없음	규칙 엔진이 DEFAULT INSERT 후 UPDATE하는 구현을 막지 않기 위함. 팩토리 메서드로 규약화 (03 §4.3). 검증은 §3 불변식 4
역방향 <code>source='RULE' ⇒ rule_id</code> 필수 없음	같은 이유 (03 §4.2)
Future 4 테이블 미생성	DR-22(확장 지점 논리 표기) · 기획서 0.3(범위 상한) 범위 상한
인덱스가 108행에서 seq scan에 짐	설계 의도 표기용. 03 §6에 명시

9. 조치 요약

항목	등급	조치	결정권
F-01 유일 제약	개선	V4__constraints.sql 1줄	팀 합의 (스키마 변경)
F-02 category CHECK	개선	같은 V4에 1줄	팀 합의 (스키마 변경)
F-03 주석 문구	문서	V1 주석 + erd.dbml 수정	DA 단독 가능
F-04 매핑 행 수	문서	erd.dbml 수정 (발표 이미지 영향)	DA 단독 가능
F-05 컬럼 순서	정보	조치 없음	—

F-01·F-02는 **판정 결과를 바꾸지 않는다**. 현재 데이터가 두 제약을 이미 만족하므로 추가해도 기존 테스트 41건이 그대로 통과한다. 다만 04 §4.10에 따라 V1을 고치지 않고 **V4를 새로 추가**해야 한다.

10. Q&A 대비 — 이 보고서에서 꺼낼 카드

예상 질문	답
"ERD가 실제 DB와 같습니까?"	컬럼 65개를 전수 대조했고, <code>ddl-auto: validate</code> 가 어긋나면 서버가 기동하지 않습니다 (§0, §2.2)
"정책을 DB에 뒀다는 게 무슨 의미입니까?"	부서 매트릭스를 표에서 옮긴 게 아니라 쿼리로 역산했고 기획서와 일치합니다 (§4.3)
"AI가 마음대로 차단할 수 있나요?"	엔티티를 우회한 직접 INSERT도 <code>SUGGESTED</code> 로 저장됩니다 (§5 테스트 1)
"확장성이 있다는 근거는?"	V3가 새 정책 축(엠바고)을 추가하며 AI 관련 컬럼을 0개 추가했습니다 (§2.4)
"정규식이 SQL에서 깨지지 않나요?"	기획서와 바이트 단위로 대조했습니다. <code>E''</code> 리터럴 0개 (§4.2)
"설계 대비 미비점은?"	5건 있고 등급·조치·결정권까지 정리했습니다 (§7, §9)

마지막 줄이 실제로는 가장 강한 카드다. 미비점을 먼저 말하는 쪽이 검증을 해 본 쪽이다.